

Fintrack

Astrosoft: Angel Villa

23 de julio de 2026

1. Introduction

1.1. Propósito de la aplicación

Fintrack es una aplicación que nos permite rastrear y analizar de forma rápida y eficiente nuestros gastos e ingresos, facilitando el desglose sobre el uso de nuestro dinero con la finalidad de mejorar la responsabilidad financiera.

1.2. Alcance y funciones principales

El alcance de Fintrack es funcionar de forma local en la ciudad para apoyar a la comunidad en la responsabilidad financiera, principalmente en un grupo reducido con el objetivo de generar consciencia sobre el uso del dinero.

2. Arquitectura de la aplicación

2.1. Diagrama Cliente-Servidor

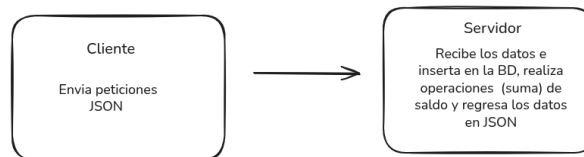


Figura 1: Diagrama Cliente Servidor

2.2. Stack tecnológico

Frontend: React Native, Expo

Backend: Node.js, JavaScript, MySQL

3. Funciones importantes

3.1. Ingreso y Gasto

```
1  const handleIngreso = async () => {
2    const monto = Number(ingreso);
3    const concepto: string = selected;
4    const sumaIngreso: number = Number(saldo) + monto;
5    const tipo: string = "ingreso";
6
7    if(!monto || monto<=0){
8      Alert.alert("Se requiere ingresar una cantidad valida");
9      return;
10   }
11   if (!concepto) {
12     Alert.alert("Categoría requerida", "Por favor selecciona un tipo de ingreso.");
13     return;
14   }
15
16   try {
17     setModalIngreso(false);
18
19     const data = await regIngreso({ tipo, concepto, id, sumaIngreso, monto });
20
21     setIngreso("");
22     setSelected("");
23     setSaldo(sumaIngreso);
24     Alert.alert("Ingreso registrado", concepto);
25   }
26   catch (error) {
27     setModalIngreso(true);
28     console.log("Error en handleIngreso:", error);
29     Alert.alert("Error", "Ocurrió un problema al registrar tu ingreso en el servidor.");
30   }
31  };
```

Figura 2: Código ingreso

La función `handleIngreso` funciona de forma asincrónica, esta función toma datos como el monto, el concepto, la suma del ingreso (el saldo actual traído de la base de datos + el monto que ingresó el usuario) y el tipo de ingreso seleccionado.

También determina si estas variables están vacías y le solicita al usuario llenar todos los campos requeridos. Posteriormente, si se validan los campos, se cierra el modal, se inicia el proceso de enviar los datos a través del servicio de backend `regIngreso`, se limpian las entradas y se muestra con éxito el ingreso registrado a través del componente `Alert` de React Native.

```

1  const handleGasto = async ()=>{
2    const monto = Number(gasto);
3
4    const concepto: string = selected;
5    const restaGasto: number = Number(saldo) - monto;
6    const tipo:string = "gasto";
7
8    if(!monto || monto<=0){
9      Alert.alert("Se requiere ingresar una cantidad valida");
10   }
11   if (!concepto) {
12     Alert.alert("Categoría requerida", "Por favor selecciona un tipo de gasto.");
13     return;
14   }
15
16   try{
17     setModalGasto(false);
18     const data = await regGasto({tipo, concepto, id, restaGasto, monto });
19     setGasto("");
20     setModalGasto(false);
21     setSelected("");
22     Alert.alert("Gasto registrado", concepto);
23   }
24   catch (error) {
25     setModalGasto(true);
26     console.log("Error en handleIngreso:", error);
27     Alert.alert("Error", "Ocurrió un problema al registrar tu ingreso en el servidor.");
28   }
29
30
31   setModalGasto(false);
32   setSaldo(restaGasto);
33   setGasto("");
34   setSelected(""); // Limpiamos la categoría seleccionada
35 };

```

Figura 3: Código gasto

De la misma forma la función `handleGasto` funciona de forma asíncrona, esta función también envía los mismos parámetros que la función ingreso y automáticamente realiza una resta.

3.2. React Native

Para esta aplicación el desarrollo se esta llevando a cabo a través del framework React Native, lo que nos da la facilidad de trabajar con tecnologías usadas en el ámbito web para programar y compilar una aplicación multiplataforma de Android y Ios. React Native nos permite trabajar con la sintaxis fuertemente tipada de Typescript y mantener una coherencia entre los tipos de datos, ademas de usar directamente componentes predefinidos por la librería de React.

Algunos de estos componentes principalmente usados son:

View: Este componente es similar a usar un div en HTML, nos permite separar en contenedores cada sección de la pantalla.

KeyboardAvoidingView: Este componente funciona como una vista que permite el desbordamiento del contenido ante el teclado, funciona con Inputs

para no tapar la información.

TextInput: Componente que permite al usuario ingresar datos de diferentes tipos.

Pressable: Similar a un Button en HTML este componente permite ejecutar una serie de instrucciones al presionar.

ScrollView: Componente que permite deslizar en la pantalla (scroll) al momento de desbordar elementos.

PieChart: Es una libreria de React Native que nos permite mostrar gráficas de pastel en pantalla recibiendo valores de value y color que actuan como parametros que ayudan a formar la gráfica.

3.3. Backend

Para este caso en especifico se diseño una arquitectura API REST que utiliza Javascript con Node y Express para crear rutas GET Y POST que nos permiten conectar lo que visualiza el usuario con la base de datos y logica de nuestra aplicación.

API: Se utiliza una API que conecta las rutas con la base de datos a traves de funciones asíncronas.

MySQL: Como motor de base de datos se opta por utilizar MySQL server al ser algo que no requiere datos abstractos y contar con tablas estructurales que organizan la información precisa, SQL cumple con los requisitos para ser utilizada sin problemas.

Services: Al contar con una API que recibe datos, desde el frontend creamos los servicios que conectaran tanto la API como las rutas para el inicio de sesión, registro y el dashboard.



Figura 4: Ejemplo de Ruta

Como se aprecia en la figura 4, se requiere hacer uso de las librerías express y pool que conectan nuestra aplicación, express nos permite levantar un puerto en el servidor que será utilizado por nuestro servicio de backend independientemente de la base de datos utilizada.

Pool por su lado nos permite crear los parámetros de conexión a nuestra base de datos, estableciendo el usuario, la base de datos, el host, etc. De esta manera se controla el servicio que une las rutas con las tablas.